Work Sheet 5 (Data Structures)

1. **Reverse a Linked List**
   Write a function to reverse a linked list. You should reverse the links between the nodes so that the last node becomes the head, and the head becomes the last node.

2. **Write the output of following. Identify issue if any**

```
#include <iostream>
using namespace std;

struct Node {
    int data;
    Node* next;
    Node(int val) : data(val), next(nullptr) {}
};

Node* createLinkedList(int size) {
    if (size <= 0) return nullptr;

    Node* head = new Node(1);
    Node* temp = head;
    Node* referencePoint = nullptr;

    for (int i = 2; i <= size; ++i) {
        temp->next = new Node(i);
        temp = temp->next;

        if (i == size / 2) {
            referencePoint = temp;
        }

        if (i == size - 1 && referencePoint) {
            temp->next = referencePoint;
        }
    }

    return head;
}

int main() {
    int size = 10;
    Node* head = createLinkedList(size);

    Node* temp = head;
    cout << "Traversing the list:" << endl;
    int count = 0;
    while (temp != nullptr && count < 20) {
        cout << "Node data: " << temp->data << endl;
        temp = temp->next;
        count++;
    }

    return 0;
}
```

3. **Merge Two Sorted Linked Lists**
   Write a function that merges two sorted linked lists into a single sorted linked list. Do not use any additional data structures; modify the links directly.
4. **Remove Duplicates from a Sorted Linked List**
   Given a sorted linked list, remove all duplicate values so that each element appears only once. Write a function to perform this in a single traversal.
5. **Find the Middle Node of a Linked List**
   Write a function to find the middle node of a linked list. If there are two middle nodes, return the second one. Use only one traversal.
6. **Reverse Every k Nodes in a Linked List**
   Given a linked list, write a function to reverse every k nodes. For example, if k = 3 and the list is 1 -> 2 -> 3 -> 4 -> 5 -> 6, it should return 3 -> 2 -> 1 -> 6 -> 5 -> 4.
7. **Partition a Linked List Around a Value x**
   Write a function to partition a linked list around a given value x such that all nodes with values less than x come before nodes with values greater than or equal to x. Preserve the original relative order of nodes in each partition.
   Original List: 3 -> 5 -> 8 -> 5 -> 10 -> 2 -> 1 -> nullptr
   Partitioned List around 5: 3 -> 2 -> 1 -> 5 -> 8 -> 5 -> 10 -> nullptr
8. **Sort a Linked List**
   Implement a function to sort a linked list. Do not convert the linked list to an array; perform the sorting directly on the linked list nodes.
9. **Clone a Linked List with Random Pointers**
   Write a function to clone a linked list where each node has an additional random pointer that can point to any node in the list or be NULL. Your function should create a deep copy of the list.